

기말 06-09장 핵심개념 빈출정리

내일 시험 직전용. 2023/2024 기말 출제 스타일 기준으로 빈출 개념을 특별 표시하고, 헛갈리는 지점을 이해 중심으로 정리했습니다.

0. 출제패턴 먼저 잡기

이 교수님 기말은 "개념을 아는지"보다 "그 개념을 코드 빈칸과 실행 결과에 정확히 적용하는지"를 묻는 쪽입니다.

빈출반복 출제된 형태

- CMP 직후 signed/unsigned flag 조건 쓰기
- C/C++ 조건문을 jump label 구조로 바꾸기
- MUL/DIV/IDIV 실행 후 레지스터, 오류 여부 판단
- swap을 C convention 또는 STDCALL로 작성/호출
- heap, stack, paging 설명 중 틀린 문장 고르기
- REAL4 bit pattern과 little endian 메모리 순서

전략점수 따는 순서

- 조건 점프 표와 MUL/DIV 표를 먼저 외운다.
- 코드 빈칸은 "반대 조건으로 탈출"인지 "참이면 이동"인지 label부터 본다.
- 나눗셈 문제는 먼저 DIV인지 IDIV인지 표시한다.
- 프로시저 문제는 [ebp+8], [ebp+12], ret/ret 8, local 변수 위치를 실수하지 않는다.

1. 06장 Conditional Processing

빈출2023: CMP signed/unsigned 설명, Boolean 문자열 변환, .WHILE 변환. 2024: JA/JB/JE/JG/JL/JNE flag 조건, JG/LOOPNE 실행 결과.

1-1. Boolean 명령어는 "결과 저장 여부"가 핵심

명령어	이해	flag	시험 포인트
AND	원하는 bit를 0으로 끄는 마스크	CF=0, OF=0, SF/ZF/PF 갱신	소문자->대문자: bit 5 clear
OR	원하는 bit를 1로 켜는 마스크	CF=0, OF=0, SF/ZF/PF 갱신	대문자->소문자: bit 5 set
XOR	1인 bit만 toggle	CF=0, OF=0, SF/ZF/PF 갱신	대소문자 서로 바꾸기
NOT	모든 bit 반전	flag 변경 없음	참거짓 선지 단골 함정
TEST	AND처럼 flag만 계산	결과 저장 없음, CF=0, OF=0	특정 bit 검사

CMP	dest - src처럼 flag만 계산	operand 변경 없음	조건 점프 직전 거의 항상 등장
-----	-----------------------	---------------	-------------------

```
; ASCII 대소문자 핵심: bit 5
and al, 11011111b ; 소문자 -> 대문자
or  al, 00100000b ; 대문자 -> 소문자
xor al, 00100000b ; 대문자 <-> 소문자 toggle
test al,00100000b ; bit 5가 1인지 검사, al은 안 바뀜
```

1-2. 조건 점프 표는 그대로 써야 한다

무조건 암기JA/JB는 unsigned, JG/JL은 signed. cmp left, right 다음 조건은 항상 left ? right 기준으로 읽습니다.

관계	unsigned jump	flag 조건	signed jump	flag 조건
>	JA	CF=0 AND ZF=0	JG	ZF=0 AND SF=OF
>=	JAE	CF=0	JGE	SF=OF
<	JB	CF=1	JL	SF!=OF
<=	JBE	CF=1 OR ZF=1	JLE	ZF=1 OR SF!=OF
==	JE/JZ	ZF=1	JE/JZ	ZF=1
!=	JNE/JNZ	ZF=0	JNE/JNZ	ZF=0

함정80h는 unsigned로 128이지만 signed byte로는 -128입니다. 그래서 같은 CMP라도 JA와 JG 결과가 달라질 수 있습니다.

1-3. C 조건문을 ASM으로 바꾸는 법

기출 빈칸은 label 흐름을 봐야 합니다. IF문은 보통 조건의 반대를 써서 false로 보냅니다.

```
; if (a <= b && c != d) x = 1; else x = 0; signed
mov eax, a
cmp eax, b
jg  L_FALSE ; a <= b의 반대는 a > b
mov eax, c
cmp eax, d
je  L_FALSE ; c != d의 반대는 c == d
mov x, 1
jmp L_END
L_FALSE:
mov x, 0
L_END:
```

```

; if (a < b || c >= d) x = 1; signed
mov eax, a
cmp eax, b
jl L_TRUE      ; OR는 하나라도 참이면 true
mov eax, c
cmp eax, d
jge L_TRUE
jmp L_END
L_TRUE:
mov x, 1
L_END:

```

1-4. LOOPNE/LOOPE

명령어	동작
LOOPE/LOOPZ	ECX를 1 감소시킨 뒤 ECX != 0이고 ZF=1이면 점프
LOOPNE/LOOPNZ	ECX를 1 감소시킨 뒤 ECX != 0이고 ZF=0이면 점프

주의 LOOPNE 자체가 ZF를 새로 만들지 않습니다. 직전의 INC, CMP, TEST 등이 만든 ZF를 그대로 봅니다.

```

; 함정 예시: LOOPNE는 ZF를 만들지 않지만, 직전 DEC가 ZF를 만든다.
mov ecx, 5
mov al, 2
L1: dec al      ; AL=0이 되는 순간 ZF=1
    loopne L1   ; ECX 감소 후 ECX!=0 AND ZF=0일 때만 반복

```

2. 07장 Integer Arithmetic

빈출2023: imul eax, 72 shift 최적화, MUL/DIV 참거짓, CDQ 직접 구현, 산술식 변환. 2024: DIV 실행 결과, 구구단 빈칸, shift 곱셈, 나머지 식 변환.

2-1. Shift와 Rotate

명령어	핵심 이해	시험 활용
SHL/SAL	왼쪽 shift, 빈 LSB는 0, 빠진 MSB는 CF	unsigned 곱하기 2^n
SHR	오른쪽 logical shift, 빈 MSB는 0	unsigned 나누기 2^n
SAR	오른쪽 arithmetic shift, sign bit 복제	signed 값을 2^n 으로 나눌 때 사용. 음수는 내림 방향에 주의
ROL/ROR	빠진 bit가 반대편으로 돌아옴	nibble 교환, bit 회전 추적

```

; var * 72 = var * (64 + 8)
mov eax, var
mov ebx, eax
shl eax, 6
shl ebx, 3
add eax, ebx

```

이해상수 곱셈 최적화는 2의 거듭제곱 합으로 분해합니다. 72는 64+8, 84는 64+16+4처럼 쪼갭니다.

SAR 함정SAR은 sign bit를 복제하므로 signed 값을 2의 거듭제곱으로 나눌 때 쓰지만, 음수에서 IDIV처럼 0 방향으로 truncate하지 않고 더 작은 정수 쪽으로 내려갈 수 있습니다. 예: $-9 \text{ sar } 1 = -5$, $-9 \text{ idiv } 2 = -4$.

2-2. MUL/IMUL 결과 위치

operand 크기	암시 피승수	결과	예
8비트	AL	AX	mul bl
16비트	AX	DX:AX	mul bx
32비트	EAX	EDX:EAX	mul ebx

- MUL은 unsigned, operand 1개, immediate 사용 불가.
- unsigned MUL: 결과의 상위 절반이 0이 아니면 CF=1, OF=1.
- 1-operand IMUL은 signed이며 결과 위치는 MUL과 같다.
- 1-operand IMUL: 결과의 상위 절반이 하위 절반의 sign extension이 아니면 CF=1, OF=1.
- 2/3-operand IMUL은 결과가 목적지 크기로 잘릴 수 있다.

함정IMUL 결과가 AX, DX:AX, EDX:EAX처럼 두 배 공간에 저장된다고 해서 CF/OF가 항상 0은 아닙니다. 예를 들어 signed byte $70 * 3 = 210$ 은 AX=00D2h이지만 하위 byte D2h의 sign extension은 FFD2h라서 overflow입니다.

2-3. DIV/IDIV는 표가 전부다

제수 크기	피제수	몫	나머지
8비트	AX	AL	AH
16비트	DX:AX	AX	DX
32비트	EDX:EAX	EAX	EDX

빈출DIV 전에는 상위 절반을 0으로 만든다. IDIV 전에는 부호 확장한다. 몫이 목적지 크기를 넘으면 divide error입니다.

```
; unsigned 32-bit division
mov eax, dividend
mov edx, 0
div ebx          ; eax=quotient, edx=remainder
```

```
; signed 32-bit division
mov eax, dividend
cdq              ; eax sign을 edx 전체로 확장
idiv ebx         ; eax=quotient, edx=remainder
```

함정 unsigned 나눗셈인데 CDQ를 쓰면 안 됩니다. CDQ는 EAX의 sign bit를 보고 EDX를 00000000h 또는 FFFFFFFFh로 만들기 때문에, unsigned 값의 최상위 bit가 1이면 피제수가 잘못 커질 수 있습니다. unsigned DIV 전에는 보통 mov edx,0입니다.

부호 확장	의미
CBW	AL -> AX
CWD	AX -> DX:AX
CDQ	EAX -> EDX:EAX

```
; VAR4 = (4 * VAR1) % (VAR2 - VAR3), signed DWORD
mov eax, VAR1
shl eax, 2
mov ebx, VAR2
sub ebx, VAR3
cdq
idiv ebx
mov VAR4, edx    ; 나머지는 EDX
```

```
; VAR4 = (4 * VAR1) % (VAR2 - VAR3), unsigned DWORD
mov eax, VAR1
shl eax, 2
mov ebx, VAR2
sub ebx, VAR3
mov edx, 0        ; unsigned DIV 전 상위 절반 clear
div ebx
mov VAR4, edx
```

실행추적 함정SHR 뒤에 SAR가 오면 처음 값의 부호가 계속 보존된다고 착각하면 틀립니다. SHR이 먼저 MSB를 0으로 만들면 그 다음 SAR는 이미 양수처럼 shift합니다.

2-4. ADC/SBB

큰 정수를 여러 조각으로 계산할 때 carry/borrow를 다음 조각으로 넘깁니다.

```
adc dest, src      ; dest = dest + src + CF
sbb dest, src      ; dest = dest - src - CF
```

3. 08장 Advanced Procedures

빈출2023: C calling convention의 swap 작성/호출. 2024: STDCALL의 swap 작성/호출. 두 해 모두 enter/leave 금지 조건이 붙었습니다.

3-1. Stack frame 위치

```
push ebp
mov  ebp, esp
sub  esp, local_size
...
mov  esp, ebp
pop  ebp
ret
```

위치	내용
[ebp]	이전 EBP
[ebp+4]	return address
[ebp+8]	첫 번째 parameter
[ebp+12]	두 번째 parameter
[ebp-4]	첫 번째 local DWORD

실수[ebp+4]는 첫 번째 인자가 아니라 return address입니다. 첫 번째 인자는 무조건 [ebp+8]부터 시작합니다.

3-2. C calling convention vs STDCALL

구분	C calling convention	STDCALL
push 순서	오른쪽 인자부터	오른쪽 인자부터
stack 정리	caller가 정리	callee가 정리
반환	ret 후 add esp,n	ret n
swap(&x,&y) 호출	push OFFSET ypush OFFSET xcall swapadd esp,8	push OFFSET ypush OFFSET xcall swap

3-3. swap 템플릿

```

; void swap(int *a, int *b)
swap PROC
    push ebp
    mov  ebp, esp
    sub  esp, 4      ; temp
    push eax
    push ebx
    push edx

    mov  eax, [ebp+8] ; eax = a
    mov  ebx, [ebp+12] ; ebx = b
    mov  edx, [eax]    ; edx = *a
    mov  [ebp-4], edx  ; temp = *a
    mov  edx, [ebx]    ; edx = *b
    mov  [eax], edx    ; *a = *b
    mov  edx, [ebp-4]
    mov  [ebx], edx    ; *b = temp

    pop  edx
    pop  ebx
    pop  eax
    mov  esp, ebp
    pop  ebp
    ret  8             ; STDCALL이면 ret 8, C convention이면 ret
swap ENDP

```

이해a, b는 int 값이 아니라 int의 주소입니다. 그래서 mov eax,[ebp+8]로 주소를 꺼내고, [eax]로 실제 int 값을 접근합니다.

local/push 함정sub esp,4 직후 [ebp-4]는 local temp입니다. 그 다음 push eax를 하면 stack은 더 아래로 내려가서 [ebp-8]에 저장됩니다. 따라서 위 순서에서는 push eax가 [ebp-4]를 덮지 않습니다.

ret 함정위 템플릿의 마지막 줄은 STDCALL이면 ret 8입니다. C calling convention 답안으로 쓰는 경우에는 ret만 쓰고, 호출한 쪽에서 add esp,8을 해야 합니다.

3-4. 배열과 LEA

- 배열은 보통 값 전체가 아니라 시작 주소를 reference parameter로 전달한다.
- LEA는 메모리를 읽는 명령이 아니라 주소 계산 결과를 레지스터에 넣는다.

```
lea eax, [esi+ecx*4+12] ; eax = esi + ecx*4 + 12, 메모리 읽기 아님
```

4. 09장 Memory Management

빈출2023/2024 모두 heap, stack, logical/linear/physical address, page fault 설명의 옳고 그름을 물었습니다.
2024에는 REAL4 저장/해석도 출제되었습니다.

4-1. Stack, Data, Heap 구분

영역	무엇이 위치하나	시험 함정
Data segment	전역 변수, static 데이터	프로시저 호출과 무관하게 존재
Stack	return address, parameters, local variables, saved registers	지역변수는 return 과정에서 stack pointer 조정으로 사라짐
Heap	동적 할당 메모리	procedure return만으로 자동 해제되지 않음

오답 선지"동적 할당된 변수와 지역 변수는 모두 stack에 위치한다"는 틀립니다. 지역변수는 stack, 동적 할당 메모리는 heap입니다.

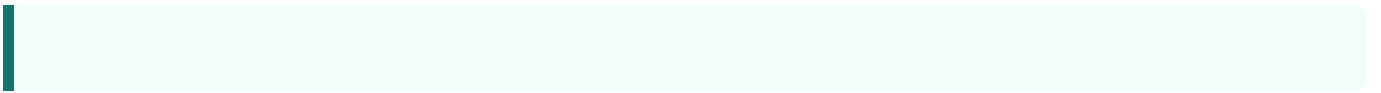
4-2. Win32 heap API

API	의미
GetProcessHeap	현재 프로세스의 기존 heap handle 반환
HeapCreate	새 private heap 생성
HeapAlloc	heap에서 memory block 할당, 포인터 반환
HeapFree	할당한 block 반환
HeapDestroy	HeapCreate로 만든 heap 제거

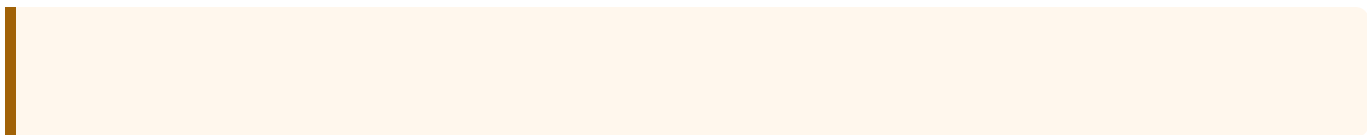
- HEAP_ZERO_MEMORY: 할당된 block을 0으로 초기화한다.
- HeapAlloc 실패 시 반환 포인터가 0인지 확인해야 한다.
- HeapFree 후 같은 포인터를 계속 쓰면 use-after-free 문제다.

4-3. Logical, Linear, Physical address

주소	뜻
Logical address	segment selector + offset
Linear address	segment descriptor의 base address + 32-bit offset
Physical address	실제 메모리 주소. paging이 켜져 있으면 linear address가 page translation을 거쳐 변환됨



순서 logical address -> segmentation -> linear address -> paging/page translation -> physical address



표현 함정 $\text{segment base} + \text{offset}$ 으로 얻는 것은 linear address입니다. paging을 사용하지 않는 경우에만 linear address가 physical address와 같다고 볼 수 있습니다.

4-4. Paging과 page fault

- page는 보통 4KB 단위다.
- $4KB = 2^{12}$ 이므로 page offset은 하위 12비트다.
- 요청 page가 메모리에 없거나 접근 권한 문제가 있으면 page fault가 발생할 수 있다.
- cache miss는 page fault가 아니다.

```
linear address = 12345ABCh
4KB page offset = 하위 12비트 = ABCh
```

4-5. REAL4 IEEE 754

필드	크기	의미
sign	1비트	0은 양수, 1은 음수
exponent	8비트	bias 127
fraction	23비트	정규화된 1.x의 x 부분

```
0.15625 = 0.00101b = 1.01b * 2^-3
sign = 0
exponent = -3 + 127 = 124 = 7Ch
fraction = 010000...
bit pattern = 3E200000h
x86 memory = 00 00 20 3E ; little endian
```

빈출REAL4 문제는 bit pattern을 구한 뒤 메모리에 적을 때 byte 순서를 뒤집는 것이 핵심입니다. 3E200000h는 낮은 주소부터 00 00 20 3E입니다.

endian 함정little endian은 byte 순서만 낮은 byte부터 저장한다는 뜻입니다. bit 순서를 뒤집는 것이 아니고, 문자열 BYTE "AB"는 낮은 주소부터 41h 42h 순서로 저장됩니다.

5. 교수님이 함정으로 내기 좋은 포인트

조건 점프signed/unsigned 섞기

- JA는 $CF=0 \text{ AND } ZF=0$, unsigned 전용처럼 생각한다.
- JG는 $ZF=0 \text{ AND } SF=OF$, signed 전용처럼 생각한다.
- 80h, FFh 같은 값은 signed와 unsigned 해석이 완전히 달라진다.

LOOPNEZF 생성 주체

- LOOPNE는 ECX를 줄이고 기존 ZF를 검사한다.
- LOOPNE 자체가 비교하거나 ZF를 새로 계산하지 않는다.
- 직전 명령이 INC/DEC/CMP/TEST인지 확인해야 한다.

DIVEDX=0 vs CDQ

- unsigned DIV r/m32: EDX:EAX 피제수, 보통 `mov edx,0`.
- signed IDIV r/m32: CDQ로 EAX -> EDX:EAX 부호 확장.
- 몫이 AL/AX/EAX에 못 들어가면 divide error.

Stackcleanup과 local 위치

- C convention: callee는 `ret`, caller가 `add esp,n`.
- STDCALL: callee가 `ret n`.
- `sub esp,4` 후 `[ebp-4]`는 local DWORD이고, 그 다음 `push`는 `[ebp-8]`부터 내려간다.

Memorycache miss와 page fault

- cache miss는 CPU cache에 없다는 뜻이다.
- page fault는 page가 메모리에 없거나 권한 문제가 있을 때 발생할 수 있다.
- 둘을 같은 개념으로 쓰는 선지는 틀린 선지로 보기 쉽다.

Endianbyte만 뒤집기

- DWORD 12345678h는 메모리에 78 56 34 12.
- REAL4 3F800000h는 메모리에 00 00 80 3F.
- bit 순서와 문자열 저장 순서를 뒤집는 것이 아니다.

SAR음수 나눗셈 반올림

- SAR은 부호 bit를 보존하는 shift라 음수에서 아래쪽으로 치우친다.
- IDIV의 몫은 0 방향으로 truncate된다.
- 음수 홀수를 2로 나누는 실행 결과 문제에서 둘을 같다고 보면 틀린다.

6. 더 출제될 만한 추가 유형

추가 대비아래 유형은 기존 2023/2024 스타일에서 조금만 변형하면 바로 출제 가능한 함정입니다. 빈칸, 실행 결과, 틀린 설명 고르기로 바뀌기 쉽습니다.

01flag가 이전 명령에서 온다

- Jcc, LOOPNE, LOOPE는 이미 만들어진 flag를 검사한다.
- MOV, LEA, LOOP, NOT은 flag를 바꾸지 않는다.
- 직전 명령이 flag를 바꾸는지부터 표시한다.

02INC/DEC와 ADD/SUB

- INC, DEC는 CF를 변경하지 않는다.

- ADD, SUB는 CF를 포함한 산술 flag를 갱신한다.
- STC 후 INC 다음 JC 같은 문제가 함정이다.

03shift 후 CF 추적

- 여러 비트 shift에서 CF는 마지막으로 밀려나간 bit다.
- SHL은 MSB 쪽에서, SHR/SAR은 LSB 쪽에서 빠진 bit를 본다.
- ROL/ROR도 마지막으로 rotate된 bit가 CF에 남을 수 있다.

04MUL immediate 불가, IMUL immediate 가능

- mul 5, div 10 같은 immediate operand는 직접 불가.
- imul eax, ebx, 5 같은 3-operand IMUL은 가능하다.
- 2023의 MUL 설명형 문제와 잘 맞는 변형이다.

058비트 DIV는 AX 전체를 나눈다

```
mov al, 100
mov ah, 1
mov bl, 10
div bl ; AX=356을 10으로 나눔
```

AL만 나눈다고 착각하면 틀립니다. 8비트 제수는 피제수가 AX입니다.

06IDIV 나머지 부호

- IDIV의 몫은 0 방향으로 truncate된다.
- 나머지 부호는 divisor가 아니라 dividend의 부호를 따른다.
- $-17 / 5$: 몫 -3, 나머지 -2.

07RET n의 n은 byte 수

- DWORD 인자 2개: ret 8.
- WORD 인자 2개: ret 4.
- 32-bit에서 pointer 인자 하나는 4바이트다.

08saved register는 역순 복원

```
push ebx
push esi
; ...
pop esi
pop ebx
```

push ebx, push esi 후 pop ebx, pop esi를 하면 값이 서로 바뀝니다.

09push와 sub esp 순서

```
push ebp
mov  ebp, esp
push ebx    ; [ebp-4] = saved EBX
sub  esp, 4  ; [ebp-8] = local DWORD
```

sub esp,4를 먼저 했는지, push ebx를 먼저 했는지에 따라 [ebp-4]의 의미가 달라집니다.

10LEA 산술 최적화

```
lea eax, [ebx+ebx*4] ; eax = ebx * 5
```

LEA는 메모리 읽기가 아니라 주소식 계산입니다. 곱셈/덧셈 최적화 문제로 나올 수 있습니다.

11page offset 계산

- 4KB page면 offset은 하위 12비트.
- 89ABCDEFh의 page offset은 DEFh.
- 상위 bit는 page number 쪽이다.

12REAL4 자주 나오는 값

- 3F800000h = 1.0, memory 00 00 80 3F.
- BF800000h = -1.0, memory 00 00 80 BF.
- 40000000h = 2.0, 3F000000h = 0.5.

13.WHILE/.IF를 jump로 바꾸기

- 반복문은 보통 조건의 반대를 써서 탈출한다.
- .WHILE eax != ebx의 탈출 조건은 JE.
- signed인지 unsigned인지 명시되면 JG/JA 계열을 구분한다.

14이중 LOOP의 ECX 보존

```
push ecx
mov  ecx, innerCount
L2: loop L2
pop  ecx
loop L1
```

inner loop도 ECX를 쓰므로 outer loop의 ECX를 저장/복원해야 합니다.

15memory-to-memory 연산 불가

- add var1, var2, cmp mem1, mem2는 일반적으로 직접 불가.

- 레지스터에 하나를 옮긴 뒤 연산해야 한다.
- `mov eax,var2 / add var1,eax`처럼 바꾼다.

7. 시험 직전 마지막 체크리스트

마지막 10분조건 점프 표, MUL/DIV 표, ret/ret 8, REAL4 little endian만 다시 보세요. 새 문제를 길게 풀기보다 실수 방지에 집중하는 쪽이 점수에 더 직접적입니다.

